



Database Management Systems

Common Table Expressions - CTE

Prof. Mahitha G

Department of Computer Science and Engineering

Acknowledgment: Teaching Assistants

Database Management Systems

Unit 2: Advanced SQL

Slides adapted from Author Slides of “Database System Concepts”, Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems”, Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Prof Mahitha G
Department of Computer Science and Engineering

Recursive CTEs

- A recursive CTE is one that references itself within that CTE.
- The recursive **CTE** is useful when working with hierarchical data as the CTE continues to execute until the query returns the entire hierarchy.
- A typical example of hierarchical data is a table that includes a list of **employees** like in the company database employee has recursive relationship, which represents the employee–supervisor relationship.
- In SQL, recursive queries are typically implemented using Common Table Expressions (CTEs) with the WITH RECURSIVE clause.

Here's the general syntax for creating a recursive query

```
WITH RECURSIVE cte_name (column_list) AS (
```

```
  -- Anchor member: The initial query that forms the base case
```

```
  SELECT ...
```

```
  FROM ...
```

```
  WHERE ...
```

```
  UNION ALL
```

```
  -- Recursive member: The query that refers to the CTE itself
```

```
  SELECT ...
```

```
  FROM cte_name -- Refer to the CTE within itself
```

```
  WHERE ...
```

```
)-- The final SELECT statement outside the CTE to retrieve the result
```

```
SELECT ...FROM cte_name;
```

Here's an explanation of each part of the syntax:

- **cte_name:** This is the name you give to the Common Table Expression (CTE), which you can use to refer to it within the CTE itself and in the final SELECT statement.
- **column_list:** Specify the list of columns you want to include in the CTE's result set.
- **Anchor member:** This is the initial query that forms the base case of the recursion. It selects the starting point or points for the recursive process. It's important to ensure that this query doesn't reference the CTE itself.

Here's an explanation of each part of the syntax:

- **UNION ALL:** This is used to combine the results of the anchor member with the results of the recursive member. UNION ALL is used because it includes duplicate rows, which is often necessary in recursive queries.
- **Recursive member:** This is the query that refers to the CTE itself, creating the recursion. It selects records based on relationships established in previous iterations. This query must reference the CTE by its name and should include a termination condition to stop the recursion.
- **Final SELECT statement:** This query retrieves the final result set from the CTE. It typically follows the recursive part of the CTE definition.

Example: Given an example of a recursive query that counts down from 5 to 1:

```
WITH RECURSIVE Countdown (n) AS (
```

```
-- Anchor member
```

```
SELECT 5
```

```
UNION ALL
```

```
-- Recursive member
```

```
SELECT n - 1
```

```
FROM Countdown
```

```
WHERE n > 1
```

```
)
```

```
-- Final SELECT statement
```

```
SELECT n FROM Countdown;
```

n
5
4
3
2
1

Example: Given an example of a recursive query that counts down from 1 to 5:

```
WITH RECURSIVE cte(n) AS (  
    -- Anchor part of the query  
    SELECT 1  
    UNION ALL  
    -- Recursive part of the query  
    SELECT n + 1 FROM cte WHERE n < 5  
)  
SELECT * FROM cte;
```

n
1
2
3
4
5

Execution Flow:

A **recursive CTE** has two parts:

1. **Anchor member** → the initial result set
2. **Recursive member** → repeatedly applied until it stops producing rows

The database engine runs it in an iterative fashion.

Run the anchor query first: `SELECT 1`

This produces the first row: 1.

Then, Apply the recursive part - SQL substitutes the current contents of cte into:

```
SELECT n + 1 FROM cte WHERE n < 5
```

From (1), since $1 < 5$, it generates: 2 which is added to the result.

Give a query to extract the organizational hierarchy beginning with a specific employee, typically the CEO, and extending downward through the reporting structure.

```
with recursive orghierarchy as(
  select ssn,fname,lname,super_ssn, 1 as level
  from employee
  where super_ssn is null
  union all
  select e.ssn,e.fname,e.lname,e.super_ssn, OH.level+1
  from employee e
  join orghierarchy OH on e.super_ssn=OH.ssn
)
select * from orghierarchy;
```

Output:

ssn	fname	lname	super_ssn	level
888665555	James	Borg	NULL	1
123456789	John	Smith	888665555	2
333445555	Franklin	Wong	888665555	2
453453453	Joyce	English	333445555	3
666884444	Ramesh	Narayan	333445555	3
987654321	Jennifer	Wallace	333445555	3
999887777	Alicia	Zelaya	333445555	3
987987987	Ahmed	Jabbar	987654321	4

8 rows in set (0.04 sec)

The **CASE** statement in **SQL** is a versatile conditional expression that enables us to incorporate **conditional logic** directly within our queries. It allows you to return specific results based on certain conditions, enabling dynamic query outputs.

- It is commonly used to create new columns based on conditional logic, provide custom values, or control query outputs based on certain conditions.
- If no condition is true then the ELSE part will be executed. If there is no ELSE part then it returns NULL.

Syntax:

```
CASE case_value  
WHEN condition THEN result1  
WHEN condition THEN result2  
...  
Else result  
END CASE;
```

Write an SQL query to display the first name and salary of each employee, along with a derived column that categorizes employees into 'High', 'Medium', or 'Low' salary levels based on the following conditions:

- Salary $\geq$ 55000 THEN 'High'
- Salary $\geq$ 40,000 THEN 'Medium' else low

Query: SELECT Fname, Salary,

CASE

WHEN Salary $\geq$ 55000 THEN 'High'

WHEN Salary $\geq$ 40000 THEN 'Medium'

ELSE 'Low'

END AS Salary_Level

FROM employee;

Fname	Salary	Salary_Level
John	30000.00	Low
Franklin	40000.00	Medium
Joyce	25000.00	Low
Ramesh	38000.00	Low
James	55000.00	High
Jennifer	43000.00	Medium
Ahmed	25000.00	Low
Alicia	25000.00	Low

8 rows in set (0.00 sec)

Write an SQL query to display the first name and salary of each employee, along with a new column called Bonus_Status that categorizes employees based on their salary as follows:

- Salary $\geq 50,000 \rightarrow$ 'Eligible for 20% Bonus'
- Salary between 40,000 and 49,999 $\rightarrow$ 'Eligible for 10% Bonus'
- Salary $< 40,000 \rightarrow$ 'No Bonus'

Expected Output:

—TRY this

Fname	Salary	Bonus_Status
John	30000.00	No Bonus
Franklin	40000.00	Eligible for 10% Bonus
Joyce	25000.00	No Bonus
Ramesh	38000.00	No Bonus
James	55000.00	Eligible for 20% Bonus
Jennifer	43000.00	Eligible for 10% Bonus
Ahmed	25000.00	No Bonus
Alicia	25000.00	No Bonus

Solution for the previous query

SELECT

Fname, Salary,

CASE

WHEN Salary >= 50000 THEN 'Eligible for 20% Bonus'

WHEN Salary >= 40000 THEN 'Eligible for 10% Bonus'

ELSE 'No Bonus'

END AS Bonus_Status

FROM employee;

Write a query to display each employee's first name, number of projects they are working on, and a new column Workload_Status as follows:

- 3 or more projects → Overloaded
- 2 projects → Normal Load
- 1 or 0 projects → Light Load

Expected Output:

Fname	Project_Count	Workload_Status
John	2	Normal Load
Franklin	4	Overloaded
Joyce	2	Normal Load
Ramesh	1	Light Load
James	1	Light Load
Jennifer	2	Normal Load
Ahmed	2	Normal Load
Alicia	2	Normal Load

Solution for the previous query

SELECT

e.Fname, COUNT(w.Pno) AS Project_Count,

CASE

WHEN COUNT(w.Pno) >= 3 THEN 'Overloaded'

WHEN COUNT(w.Pno) = 2 THEN 'Normal Load'

ELSE 'Light Load'

END AS Workload_Status

FROM employee e

LEFT JOIN works_on w ON e.Ssn = w.Essn

GROUP BY e.Fname;

The same query using a CTE to show how CASE can be combined with CTE for better readability

```
WITH Project_Counts AS (  
    SELECT e.Fname, COUNT(w.Pno) AS Proj_Count  
    FROM employee e LEFT JOIN works_on w ON e.Ssn = w.Essn  
    GROUP BY e.Fname  
)  
SELECT Fname, Proj_Count,  
    CASE  
        WHEN Proj_Count >= 3 THEN 'Overloaded'  
        WHEN Proj_Count = 2 THEN 'Normal Load'  
        ELSE 'Light Load'  
    END AS Workload_Status  
FROM Project_Counts;
```



THANK YOU

Mahitha G

Department of Computer Science and Engineering

mahithag@pes.edu